Homework 6 Conceptual

CSCI 1470/2470: Deep Learning

Due Monday, April 19th, 2024 at 6:00pm EST

1 Conceptual Questions

1. Question 1: Show that, for some discrete distributions P and Q , the difference between the KL-Divergence and Cross-Entropy loss can be marginal. In what scenario could this happen?

[ANSWER] Cross-Entropy of two discrete probability distributions P and Q over the same probability space $\{x_1, \ldots, x_n\}$ is defined as:

$$H(P, Q) = -\sum_{i=1}^{n} P(x_i) \log Q(x_i)$$

This measures the expected number of bits required to identify an event from a set of possibilities if a coding scheme used for the set is optimized for an estimated probability distribution Q, rather than the true distribution P.

KL-Divergence (Kullback-Leibler Divergence) provides a measure of how one probability distribution diverges from a second, expected probability distribution. For distributions P and Q it is given by:

$$D_{KL}(P \parallel Q) = \sum_{i=1}^{n} P(x_i) \log \frac{P(x_i)}{Q(x_i)}$$

The KL-Divergence can be expressed as the difference between the Cross-Entropy and the Entropy of P:

$$D_{KL}(P \parallel Q) = H(P, Q) - H(P)$$

where $H(P)$ is the Entropy of P defined as:

$$H(P) = -\sum_{i=1}^{n} P(x_i) \log P(x_i)$$

To show that the difference between KL-Divergence and Cross-Entropy loss can be marginal, consider:

$$H(P, Q) - D_{KL}(P \parallel Q) = H(P)$$

Here, if $H(P, Q)$ and $D_{KL}(P \parallel Q)$ are close, it implies that the entropy $H(P)$ is small. This can occur under a few scenarios:

1. P and Q are similar: If P and Q are very close to each other, then $\frac{P(x_i)}{Q(x_i)} \approx 1$ and thus $\log \frac{P(x_i)}{Q(x_i)} \approx 0$. Hence, $D_{KL}(P \parallel Q)$ becomes very small. In this case, $H(P, Q)$ is only slightly higher than $H(P)$ because the extra bits required due to the wrong model Q are minimal.

2. Both distributions are highly skewed towards certain outcomes: If both distributions assign most of their probability mass to a few outcomes, and these outcomes coincide, then both $H(P)$ and $D_{KL}(P \parallel Q)$ will be small because there is little uncertainty in both the true and the model distributions.

In Conclusion, in scenarios where the distributions P and Q are closely aligned, especially in cases where both distributions are highly skewed towards certain outcomes, the difference between the Cross-Entropy and the KL-Divergence is minimal. As demonstrated, when the true distribution P and the model distribution Q both assign high probabilities to the same outcomes, the entropy $H(P)$ is low, and the additional uncertainty introduced by using Q instead of P (as quantified by $D_{KL}(P \parallel Q)$) is minimal.

2. Question 2: Your friend has been developing a multimodal model, where they take in multiple types of input streams at the same time and reason about them in a unified architecture. Specifically, they try to figure out a per-frame classification based on both an image and a frame of audio, both of which are sampled from a large video corpus.

   Their network is performing about the same as it did with audio alone, but way better than with images alone. You're curious how well the image input is propagating through the network and thereby contributing to the model prediction.

   How can you leverage content from generative models (i.e. reconstruction) to try to answer that question? Furthermore, if you find that the image data is not propagating meaningfully through the network, can you try to force the image information to propagate further (i.e. by incorporating something into the loss)?

   [ANSWER]
   We can train an auxiliary generative model, such as a Variational Autoencoder (VAE) or a Generative Adversarial Network (GAN), to reconstruct the input image from intermediate layer activations of the multimodal model. By examining how accurately the reconstructed images match the original input, you can infer how much image information is retained or lost as it propagates through the network. For instance, if the reconstruction from deeper layers still retains critical features of the original images, it suggests that the image information is propagating effectively.

3. Question 3 : One common problem encountered during GAN training is mode collapse. Explain what mode collapse is and why it occurs. (2-4 sentences)

[**ANSWER**] Mode collapse is a phenomenon in Generative Adversarial Networks (GANs) where the generator starts producing a limited variety of outputs, often very similar to each other, despite being trained on a diverse dataset. This occurs because the generator finds a particular pattern or subset of data that consistently fools the discriminator into thinking that the outputs are real, leading it to focus on these successful outputs and neglect others. As a result, the diversity of the generated samples decreases, and the generator fails to represent the full complexity of the target distribution. This issue is often exacerbated by the adversarial nature of the training process, where the generator and discriminator are in a continuous loop of trying to outsmart each other.

4. Question 4 : While training your GAN model, you notice that the discriminator rapidly converges to near 100 percent accuracy and your generator accuracy doesn't improve over time. What kinds of things could be causing this? List a few possible contributing factors. (2-4 sentences)

[ANSWER] When training a Generative Adversarial Network (GAN) and observing that the discriminator converges too quickly to high accuracy while the generator stalls, several factors could be at play:
1. Imbalance in Model Capacity: If the discriminator is too powerful compared to the generator, it may easily distinguish real from fake data without giving the generator a chance to improve. Balancing the capacities of these models is crucial.
2. Poor Generator Initialization: The initial parameters of the generator might be inadequate, leading to poor quality samples that fail to evolve over time. Better initialization strategies or different architectural choices might be necessary.
3. Inadequate Training Procedure: Issues such as using inappropriate learning rates, batch sizes, or optimizer settings can disproportionately favor the discriminator. Adjusting these training parameters could help provide a more balanced learning environment.
4. Vanishing Gradient Problem: If the discriminator gets too good, the generator may receive gradients that are too small to make meaningful updates, a scenario known as the vanishing gradients problem. Implementing techniques like modified loss functions or gradient penalty can help mitigate this issue.

2 2470-only Questions

1. Question 1: For VAEs, why do we have to maximize the lower bound of the log-likelihood? When would this be problematic?

[ANSWER] Why Maximize the Lower Bound? The true data log-likelihood, $\log p_\theta(x)$, where x is the observed data and θ represents the parameters of the model, is generally intractable in models like VAEs. This intractability arises because it involves an integral over the latent variables z that cannot be computed analytically due to the complexity of the integrand.To overcome this, VAEs introduce a variational approximation to the posterior distribution $p_\theta(z|x)$, denoted as $q_\phi(z|x)$, where ϕ are the parameters of this approximate distribution. The variational lower bound (also known as the ELBO - Evidence Lower Bound) is defined as:

$$\mathcal{L}(\theta, \phi; x) = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - D_{KL}(q_\phi(z|x) \| p_\theta(z))$$

This bound is a sum of the expected log-likelihood of the data given the latent variables under the approximate posterior and the negative Kullback-Leibler divergence between the approximate posterior and the prior on the latent variables. Maximizing this lower bound effectively pushes the approximate posterior $q_\phi(z|x)$ to be similar to the true posterior $p_\theta(z|x)$, while also trying to maximize the likelihood of the data under the model. Since $\log p_\theta(x) \geq \mathcal{L}(\theta, \phi; x)$, maximizing $\mathcal{L}$ indirectly maximizes $\log p_\theta(x)$.
Potential Problems with Maximizing the Lower Bound
While maximizing the ELBO is practical and generally effective, it can pose certain challenges:
1. Model Bias: The choice of the family of distributions for $q_\phi(z|x)$ can impose limitations. If the variational family is too restrictive (e.g., choosing a Gaussian with a diagonal covariance matrix), it may not capture the true posterior adequately. This restriction leads to a gap between the true log-likelihood and the ELBO, which can bias the model.
2. Local Optima: Like many optimization problems in high-dimensional spaces, training VAEs can converge to local optima. This issue is exacerbated by the non-convexity of the objective function with respect to the parameters θ and ϕ.
3. High Variance in Gradient Estimates: The reparameterization trick, used to allow gradient-based optimization through the ELBO, can lead to high variance in gradient estimates, especially if the latent space is not well-behaved or is poorly approximated by $q_\phi(z|x)$.
4. Ignoring the Structure in Latent Space: Maximizing the ELBO focuses on matching the approximate and true posteriors and does not necessarily encourage the learning of useful or interpretable latent representations, which might be critical for some applications.

2. Question 2: One intriguing application of GANs is to "translate" existing images into their "artistic" counterparts. Similar features have even been launched in Photoshop (see this video). Check out this CycleGAN project page, and answer: how do the authors manage to train conditioned GANs without paired images? (2-3 sentences)

   [ANSWER]
   The authors of CycleGAN manage to train conditioned GANs without paired images by using a method called cycle consistency loss. This approach allows the model to learn how to translate between two domains without having explicit paired examples showing the direct transformation from one domain to another. Instead, the model uses two generators and two discriminators to enforce that an input image from one domain can be translated to another domain and then back again, ending up close to the original image. This cycle consistency ensures that the model captures the underlying characteristics of both domains effectively and maintains meaningful translations, even in the absence of paired data.

3. Question 3: Regular GAN's suffer from the vanishing gradient problem. One attempt to alleviate this issue is using Wasserstein loss. What is Wasserstein loss, and how is it different from the regular GAN loss function? Why does it not suffer as badly from vanishing gradients, and is there any special considerations that need to be made when implementing it (i.e. enforcing some constraint)? Feel free to read up on this method on your own to complement the lecture material.

[ANSWER]

Standard GAN Loss: The standard GAN framework involves a generator G and a discriminator D. The discriminator tries to distinguish real data from fake data generated by G, while G tries to fool D. The loss function typically used is the cross-entropy loss, which measures the performance of a classification model whose output is a probability value between 0 and 1. The loss functions for the discriminator and generator are defined as:

$$L_D = -\mathbb{E}_{x\sim\text{real data}}[\log D(x)] - \mathbb{E}_{z\sim p(z)}[\log(1 - D(G(z)))]$$

$$L_G = -\mathbb{E}_{z\sim p(z)}[\log D(G(z))]$$

Wasserstein Loss: Instead of classifying the outputs as real or fake, the discriminator in a Wasserstein GAN (often called a critic) assesses how "real" the distribution of the generated data is compared to the real data. The critic is trained to produce a score (not a probability) that indicates the "realness." The Wasserstein loss aims to minimize the Earth Mover's (or Wasserstein) distance between the distribution of the real data and the generated data, which intuitively represents the cost of transporting mass to transform the generated distribution into the real distribution. The loss functions for the discriminator (critic) and generator in the Wasserstein GAN are:

$$L_C = \mathbb{E}_{x\sim\text{real data}}[C(x)] - \mathbb{E}_{z\sim p(z)}[C(G(z))]$$

$$L_G = -\mathbb{E}_{z\sim p(z)}[C(G(z))]$$

Advantages Regarding Gradients:

Wasserstein GANs help mitigate the vanishing gradient problem by using a loss function that provides more meaningful gradients throughout training. In standard GANs, when the discriminator gets too good, the generator's gradients can vanish, making it hard to improve further. However, the Wasserstein loss gives a gradient that is more stable and useful for training the generator because it reflects the actual distance between the distributions of real and generated samples. This distance remains meaningful and differentiable more consistently throughout training.

Implementation Considerations:

One crucial aspect of implementing Wasserstein GANs is enforcing a constraint on the critic to keep it within the space of 1-Lipschitz functions. This is usually achieved by:

Weight Clipping: The weights of the critic are clipped to a small range, typically $[-c, c]$. However, this method can lead to other issues, such as limiting the capacity of the critic and causing erratic training behavior.

Gradient Penalty (GP): A more stable method involves adding a penalty on the gradient norm for random points between real and generated data. This penalty ensures that the gradients do not grow too large, which helps maintain the Lipschitz constraint more reliably.

In conclusion, the Wasserstein loss provides a robust way to train GANs, addressing some inherent problems in the traditional approach. It offers smoother training dynamics and more meaningful feedback for the generator, reducing the likelihood of vanishing gradients. Implementing it with the gradient penalty method rather than weight clipping tends to yield better results, providing a more stable and effective training process.